Universidad de los Andes | FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

INFORME PROYECTO 2

Juego de Reflejos en Assembly sobre Pochoco SoC

Instanciación del Pochoco SoC en FPGA e implementación del juego en assembly
Lattice iCE40 HX1K — Nandland Go Board

Integrantes: Lucas Nestler, Santiago Arnolds, Matías Veto
Curso: Arquitectura de Computadores, semestre 2026-2
Profesor: Jorge Gómez Mir

Entrega en GitHub: 29 de septiembre · Demostración en clases: 30 de septiembre

Índice

1. Arquitectura del juego, contador de ciclos y temporización de 3 s **3**

1.1 Diagrama de bloques 3

1.2 Mapa de memoria usado por el juego 4

1.3 Contador de ciclos 4

1.4 Frecuencia de reloj y conversión ciclos → tiempo 4

1.5 Secuencia de una ronda (arquitectura del juego) 5

1.6 Generación pseudoaleatoria 5

2. Programa en assembly: medición de tiempo y promedio de diez rondas **6**

2.1 Convención de registros 6

2.2 Medición del tiempo de respuesta 6

2.3 Conversión a décimas de segundo y cálculo del promedio 7

2.4 Despliegue en los 7 segmentos sin división/multiplicación 7

2.5 Manejo de pulsación incorrecta 8

3. Assembler propio: codificación, etiquetas, validación y pruebas **8**

3.1 Sintaxis aceptada 8

3.2 Instrucciones soportadas 9

3.3 Tratamiento de etiquetas 9

3.4 Validación y mensajes de error 9

3.5 Ejemplo de traducción: assembly → código máquina 10

3.6 Pruebas 10

4. Validación en la FPGA y ajustes finales **11**

4.1 Reset de encendido: la CPU arrancaba antes que la RAM 11

4.2 Tiempo de cada ronda visible en pantalla 11

4.3 Reinicio de la partida con un botón 11

4.4 Cronómetro iniciado al apagarse los 4 LEDs 12

4.5 Uso de recursos y temporización 12

4.6 Pruebas en la placa 13

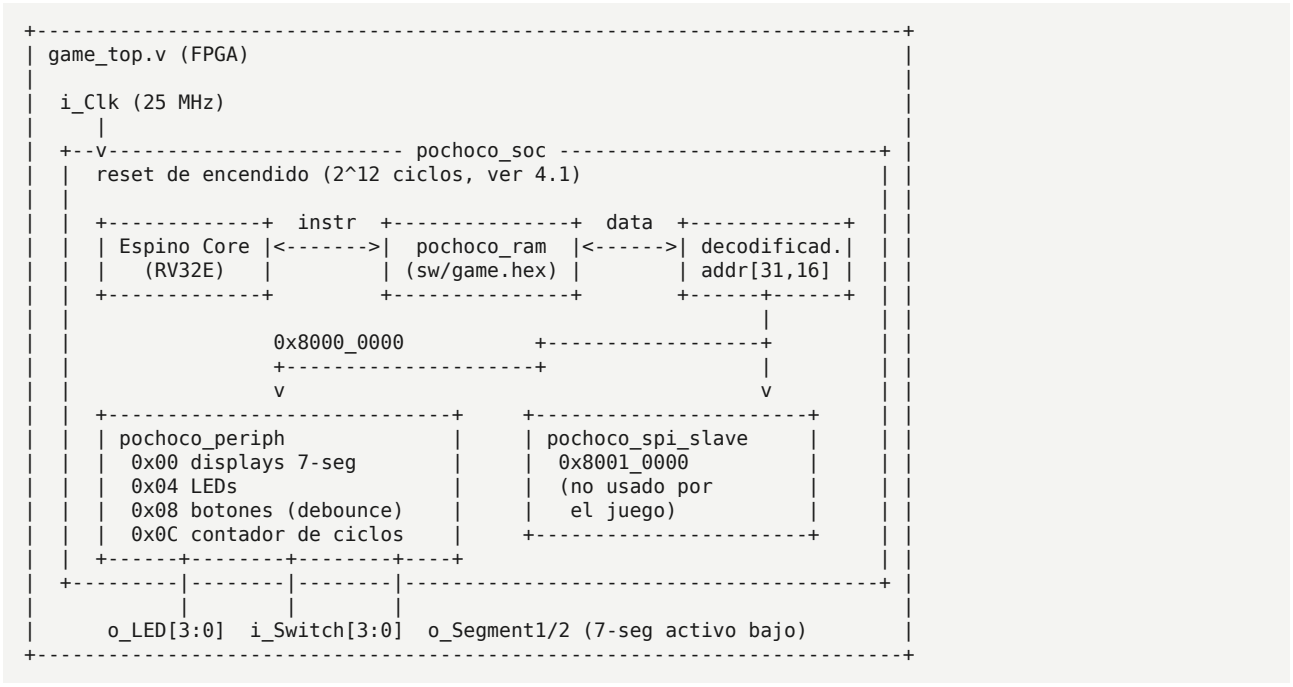
5. Conclusiones **13**

1. Arquitectura del juego, contador de ciclos y temporización de 3 s

El punto de partida es el repositorio base `pochoco_soc`: el Espino Core (pipeline RV32E: fetch, decode, ALU, banco de registros, load/store y controlador) conectado en Pochoco SoC a una RAM unificada de instrucciones/datos y a periféricos mapeados en memoria. Sobre esa base se agregó un módulo superior propio, `rtl/game_top.v`, que instancia `pochoco_soc` seleccionando `sw/game.hex` como imagen de memoria y conectando directamente los pines físicos de la Go Board (LEDs, switches, displays de 7 segmentos y SPI) según `goboard.pcf`. No se agregó lógica adicional en `game_top.v` a propósito: toda la lógica del juego vive en el software (`sw/game.s`). En el hardware sólo se agregaron el contador de ciclos y el debounce de botones en el periférico (§1.3) y se alargó el reset de encendido del SoC (§4.1); el camino de datos del núcleo no se modificó.

1.1 Diagrama de bloques

El siguiente diagrama resume cómo `game_top` se apoya en `pochoco_soc` y en los periféricos relevantes para el juego.



1.2 Mapa de memoria usado por el juego

Dirección / offset	Registro	Acceso	Descripción
`0x0000_0000`	RAM unificada	R/W	Instrucciones y datos (`game.hex` se carga aquí al boot).
`0x8000_0000 + 0x00`	Displays 7-seg	W	Byte con dos nibbles, decodificados por `hex2seg` (activo bajo en la placa).
`0x8000_0000 + 0x04`	LEDs	W	4 bits, uno por LED (one-hot para el LED objetivo, 0xF para los 4 encendidos).
`0x8000_0000 + 0x08`	Botones	R	4 bits ya filtrados (`debounce_bank`); botón SW*i* asociado al LED D*i*.
`0x8000_0000 + 0x0C`	Contador de ciclos	R/W	32 bits, incrementa en cada ciclo de reloj desde reset. Escribir un valor recarga el contador a ese valor (0 lo reinicia). Es la única fuente de tiempo del juego.

1.3 Contador de ciclos

El contador de ciclos vive en `rtl/pochoco_periph.v` (offset 0x0C), como un registro de 32 bits que se incrementa en cada flanco de reloj desde el reset del SoC. Al no requerir ninguna instrucción de temporizador dedicada, el programa lo usa como única fuente de tiempo: leerlo entrega el conteo actual, y escribirle cualquier valor lo recarga a ese valor. El programa nunca escribe al offset 0x0C: para medir un intervalo guarda el valor leído al comienzo en un registro y resta el valor leído al final, sin reiniciar el contador global. Con 32 bits a 25 MHz el contador da la vuelta cada ~172 s; como la resta es en módulo 2^{32}, un intervalo que cruce ese punto se sigue midiendo correctamente.

```
always @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) cyc_q <= 32'b0;
  else if (access & we_i & (off == 6'd3)) cyc_q <= wdata_i;   // escritura recarga
  else cyc_q <= cyc_q + 32'd1;                                // incrementa cada ciclo
end
```

1.4 Frecuencia de reloj y conversión ciclos → tiempo

El reloj de la Go Board es de 25 MHz (pin 15 en `goboard.pcf`), es decir 25 000 000 ciclos por segundo. De ahí las constantes de temporización que usa `game.s`:

Magnitud	Cálculo	Ciclos	Uso en el programa
3 segundos	`3 s × 25 000 000 ciclos/s`	`75 000 000`	Espera de los 4 LEDs encendidos al inicio de cada ronda (`wait_3s`).
2 segundos	`2 s × 25 000 000 ciclos/s`	`50 000 000`	Tiempo que el resultado de cada ronda queda visible (`hold_result`, §4.2).
1 décima de segundo	`0.1 s × 25 000 000 ciclos/s`	`2 500 000`	Unidad para convertir el tiempo de reacción (`response_div`) y para la pausa de error.

Estas constantes no caben en el inmediato de 12 bits con signo de `addi` (rango −2048...2047), así que se cargan en dos instrucciones: `lui` fija los 20 bits altos y `addi` suma los 12 bajos. Como `addi` extiende el signo de su inmediato, cuando los 12 bits bajos son ≥ 0x800 hay que redondear hacia arriba la parte de `lui` y sumar un valor negativo. Por ejemplo, 75 000 000 = 0x0478_68C0; los 12 bits bajos (0x8C0 = 2240) exceden 2047, por lo que se usa `lui t1, 0x4787` (0x0478_7000) seguido de `addi t1, t1, -1856` (0x0478_7000 − 0x740 = 0x0478_68C0). Del mismo modo, 2 500 000 = 0x0026_25A0 = `lui 0x262` + `addi 1440` y 50 000 000 = 0x02FA_F080 = `lui 0x2FAF` + `addi 128`.

1.5 Secuencia de una ronda (arquitectura del juego)

- **Inicio de ronda:** se limpia el display y se lee el contador de ciclos en a2; este valor sólo se usa para medir la espera de 3 s y como entrada del generador pseudoaleatorio.
- **Fase de 3 s:** se escriben los 4 bits de LED en 1 (0xF) y se hace polling del contador de ciclos hasta que `ciclos_transcurridos ≥ 75 000 000`.
- **Selección pseudoaleatoria:** terminada la espera, se calcula el índice 0..3 del LED objetivo (§1.6) y se escribe la máscara one-hot correspondiente en el registro de LEDs, apagando los otros tres.
- **Inicio del cronómetro:** inmediatamente después de encender sólo el LED objetivo se vuelve a leer el contador en a2 (`cyc_start`). El tiempo de reacción se mide desde aquí, es decir, desde que se apagan los 4 LEDs; los 3 s de espera no se cuentan (§2.2 y §4.4).
- **Espera de pulsación:** primero se espera a que el jugador suelte cualquier botón que hubiese quedado presionado (evita falsos aciertos instantáneos), luego se hace polling del registro de botones hasta ver un valor distinto de 0.
- **Verificación:** se relee el contador y se resta `cyc_start` para obtener el tiempo de reacción en ciclos. Si el botón coincide con la máscara objetivo, la ronda es correcta; si no, se muestra “EE”, se apagan los LEDs, se espera una pausa de error y se reinicia sólo la ronda actual (las rondas correctas y la suma de tiempos no se tocan).
- **Cierre de ronda correcta:** el tiempo se convierte a décimas de segundo, se acumula para el promedio y se muestra en los displays durante 2 s. Al llegar a 10 rondas correctas se calcula y muestra el promedio, que queda en pantalla hasta que se presiona cualquier botón; eso reinicia una partida nueva (§4.3).

1.6 Generación pseudoaleatoria

El enunciado permite usar “una semilla tomada del contador de ciclos y operaciones de assembly”. Como la ALU tiene los corrimientos (SLL/SRL/SRA/SLLI/SRLI/SRAI) deshabilitados en hardware (ejecutan como ADD, ver §2.4), no es viable un LFSR clásico basado en shifts. En su lugar, s1 se mantiene como un estado persistente entre rondas que se mezcla cada ronda con: (a) los ciclos medidos al terminar la espera de 3 s (t2), (b) una constante XOR fija (1783) para romper simetrías, y (c) el instante de inicio de la ronda (a2). El índice del LED objetivo se obtiene con `andi a3, s1, 3` (1 de 4, sin necesitar shift porque 4 es potencia de 2).

La fuente real de variabilidad es (c): el instante en que empieza cada ronda depende de cuánto tardó el jugador en todas las rondas anteriores, lo que varía entre rondas y entre partidas. El término (a) aporta poco, porque el polling termina siempre a pocos ciclos del umbral. La semilla inicial al bootear es el valor del contador en ese instante. Así se cumple el requisito de que la selección “no puede ser una secuencia fija que se repita exactamente en cada partida”; el testbench de extremo a extremo (`sim/tb_game_smoke.v`) valida que se observe más de un LED objetivo distinto en una misma partida simulada.

Limitación conocida: tras encender la placa, la primera ronda parte de una semilla casi constante (el número de ciclos entre el reset y `lw s1, 12(s0)` en `_start` es fijo), por lo que el primer LED objetivo tiende a repetirse entre encendidos. A partir de la segunda ronda la mezcla con el tiempo humano rompe ese patrón, y una partida reiniciada con un botón (§4.3) toma su semilla de un contador que ya avanzó una cantidad variable de ciclos. No afecta el requisito (variedad perceptible dentro de la partida), pero se documenta.

2. Programa en assembly: medición de tiempo y promedio de diez rondas

El programa completo vive en `sw/game.s` (224 líneas, RV32E, sin corrimientos ni multiplicación/división). No usa pila ni llamadas anidadas: una única subrutina, `show_decimal`, se invoca con `jal ra, ...` y retorna con `jalr x0, 0(ra)`.

2.1 Convención de registros

Registro	Uso
`s0`	Base de periféricos, fija en 0x8000_0000 durante todo el programa.
`s1`	Estado persistente del generador pseudoaleatorio (sobrevive entre rondas).
`a0`	`correct_count`: rondas correctas acumuladas (0..10).
`a1`	`sum_decimas`: acumulador de décimas de segundo para el promedio.
`a2`	Contador de ciclos de referencia: primero el inicio de la ronda (espera de 3 s), luego `cyc_start` (instante en que se enciende el LED objetivo). También se reutiliza en las pausas.
`a3 / a4`	Índice (0..3) y máscara one-hot del LED objetivo de la ronda actual.
`a5, t0-t2, ra`	Registros de trabajo, reutilizados libremente entre fases.

2.2 Medición del tiempo de respuesta

El tiempo de reacción se mide con el contador de ciclos: justo después de escribir la máscara del LED objetivo (que apaga los otros tres LEDs) se guarda el contador en a2. Cuando el jugador presiona un botón se vuelve a leer el contador y se resta a2, obteniendo los ciclos transcurridos desde que se apagaron los 4 LEDs; es decir, sólo el tiempo de reacción, sin la ventana fija de 3 s (ver la discusión de esta decisión en §4.4). Extracto relevante:

```
target_ready:
    sw   a4, 4(s0)          # enciende SOLO el LED objetivo (apaga los otros 3)
    lw   a2, 12(s0)         # a2 = cyc_start: el cronometro arranca AQUI
wait_release:
    lw   t0, 8(s0)
    bne  t0, x0, wait_release
wait_button:
    lw   t0, 8(s0)
    beq  t0, x0, wait_button
    lw   t1, 12(s0)
    sub  t1, t1, a2         # t1 = ciclos de reaccion
```

Entre las dos lecturas del contador sólo hay instrucciones de polling, por lo que el error de medición es de unos pocos ciclos (décimas de microsegundo), despreciable frente a la resolución de 0,1 s que se muestra. El debounce de botones (~10 ms) agrega como máximo un retardo de ese orden.

2.3 Conversión a décimas de segundo y cálculo del promedio

Sin instrucción de división en la ALU, la conversión ciclos → décimas de segundo se calcula por resta repetida: se resta 2 500 000 (1 décima a 25 MHz) del total de ciclos tantas veces como sea posible, contando las restas en a5. El resultado se trunca a la décima inferior. El mismo patrón se reutiliza para el promedio final (dividir `sum_decimas` entre 10) y, dentro de `show_decimal`, para separar decenas y unidades.

```
    # decimas de segundo = t1 / 2,500,000 (resta repetida)
    lui  t2, 0x262
    addi t2, t2, 1440        # t2 = 2,500,000 ciclos (1 decima a 25 MHz)
    addi a5, x0, 0
response_div:
    blt  t1, t2, response_div_done
    sub  t1, t1, t2
    addi a5, a5, 1
    jal  x0, response_div
response_div_done:
    add  a1, a1, a5          # sum_decimas += decimas de esta ronda
    addi a0, a0, 1           # correct_count += 1
```

Tras mostrar el tiempo de la ronda y mantenerlo 2 s en pantalla (§4.2), si `a0 == 10` el programa salta a `all_rounds_done` y aplica la misma resta repetida para dividir a1 entre 10, obteniendo el promedio en la misma unidad (décimas de segundo). El promedio se muestra con `show_decimal` y queda fijo en pantalla hasta que el jugador presiona un botón para iniciar otra partida (§4.3).

2.4 Despliegue en los 7 segmentos sin división/multiplicación

`show_decimal` recibe en a5 un valor de 0 a 99 (se recorta a 99 si se excede) y lo separa en decenas/unidades con resta repetida de 10. El periférico de 7 segmentos (`pochoco_periph.v`) ya decodifica un byte "nibble alto = dígito izquierdo, nibble bajo = dígito derecho" a los patrones de segmentos vía la función hex2seg; para reutilizarla sin instrucción de shift, el nibble de decenas se coloca en la posición alta sumando 16 tantas veces como decenas haya (equivalente a "decenas << 4", hecho con `addi t0, t0, 16` en un bucle):

```
tens_div:                    # separa decenas de unidades (resta repetida de 10)
    blt  a5, t0, tens_div_done
    sub  a5, a5, t0
    addi t1, t1, 1           # t1 = decenas
    jal  x0, tens_div
tens_div_done:
    ...
mul16:                       # decenas*16 sin shift (t0 = decenas*16)
    beq  t2, t1, mul16_done
    addi t0, t0, 16
    addi t2, t2, 1
    jal  x0, mul16
mul16_done:
    add  t0, t0, a5          # t0 = decenas*16 + unidades
    sw   t0, 0(s0)           # escribe ambos digitos al periferico de 7-seg
```

Nota: el Espino Core implementa RV32E, pero las instrucciones de corrimiento (SLL/SRL/SRA/SLLI/SRLI/SRAI) están deshabilitadas en la ALU física para ahorrar LUTs en la FPGA objetivo (iCE40 HX1K) y se ejecutan como ADD si se usan por error. Tampoco hay multiplicación ni división (RV32E no incluye la extensión M). Por eso `game.s` las reemplaza por las sumas/restas repetidas mostradas arriba.

2.5 Manejo de pulsación incorrecta

Ante un botón que no coincide con la máscara objetivo (a4), el programa apaga los LEDs, escribe 0xEE en los displays (“E E” vía el decodificador hexadecimal existente), espera una pausa visible de 5 décimas de segundo (5 × 2 500 000 ciclos) y salta de nuevo a `round_start`. Como `a0` (`correct_count`) y `a1` (`sum_decimas`) no se tocan en esta ruta, sólo se reinicia la ronda actual, tal como exige el enunciado.

3. Assembler propio: codificación, etiquetas, validación y pruebas

El assembler (`sw/assembler.py`, ~510 líneas) está escrito en Python 3 puro, sin dependencias externas y sin invocar ningún assembler de RISC-V externo en ningún punto de su ejecución: toda la codificación de instrucciones a palabras de 32 bits se implementa a mano siguiendo el formato RV32I/E estándar. Recibe un archivo `.s` y produce un `.hex` con una palabra de 32 bits en hexadecimal minúscula por línea, formato consumido directamente por `$readmemh` en `pochoco_ram.v`.

```
$ python3 sw/assembler.py sw/game.s sw/game.hex
# (sin salida = compilacion limpia; los warnings, si los hay, van a stderr)
```

3.1 Sintaxis aceptada

- **Comentarios:** todo lo que sigue a # en una línea se descarta.
- **Etiquetas:** identificador que empieza con letra, ‘_’, ‘.’ o ‘$’ seguido de letras/dígitos/‘_’/‘.’/‘$’, terminado en ‘:’. Pueden ir varias etiquetas en la misma línea (`a: b: addi x1, x0, 1`) y pueden compartir línea con una instrucción.
- **Directivas soportadas:** `.text`, `.section .text`, `.global`/`.globl`, que no ocupan espacio en la imagen; cualquier otra directiva (`.data`, `.word`, `.align`, …) se rechaza explícitamente en vez de ignorarse en silencio, porque cambiaría la salida sin que el usuario lo note.
- **Registros:** por número `x0..x15` o por nombre ABI (`zero, ra, sp, gp, tp, t0-t2, s0/fp, s1, a0-a5`). Nombres ABI que existen en RV32I pero no en RV32E (`a6, a7, s2-s11, t3-t6`) se reconocen para poder emitir un error indicando a qué registro `x` corresponden, en vez de un “instrucción desconocida” confuso.
- **Inmediatos:** decimales, hexadecimales (`0x...`), binarios (`0b...`) u octales, con signo opcional.
- **Memoria:** `offset(registro)`, con offset opcional (por defecto 0), usado por `lw/lb/lh/lbu/lhu/sw/sb/sh/jalr`.

3.2 Instrucciones soportadas

Formato	Mnemónicos	Operandos	Codificación (campos)
R	`add, sub, sll, slt, sltu, xor, srl, sra, or, and`	rd, rs1, rs2	`funct7|rs2|rs1|funct3|rd|opcode(0x33)`
I (aritm.)	`addi, slti, sltiu, xori, ori, andi`	rd, rs1, imm	`imm[11:0]|rs1|funct3|rd|opcode(0x13)`
SHIFT	`slli, srli, srai`	rd, rs1, shamt	`Igual a I, imm = funct7<<5 | shamt (0..31)`
LOAD	`lb, lh, lw, lbu, lhu`	rd, imm(rs1)	`Igual a I, opcode 0x03`
S	`sb, sh, sw`	rs2, imm(rs1)	`imm[11:5]|rs2|rs1|funct3|imm[4:0]|opcode(0x23)`
B	`beq, bne, blt, bge, bltu, bgeu`	rs1, rs2, etiqueta	`imm[12|10:5]|rs2|rs1|funct3|imm[4:1|11]|opcode(0x63)`
U	`lui, auipc`	rd, imm20	`imm20|rd|opcode(0x37/0x17)`
J	`jal`	rd, etiqueta	`imm[20|10:1|11|19:12]|rd|opcode(0x6f)`
JALR	`jalr`	rd, imm(rs1)	`Igual a I, opcode 0x67`

Todas las instrucciones usadas por `game.s` pertenecen a esta lista; no se usan corrimientos porque la ALU los ejecuta como ADD (ver §2.4).

3.3 Tratamiento de etiquetas

El assembler resuelve etiquetas en dos pasadas. La primera (`find_labels`) recorre el código llevando una dirección que arranca en 0 y avanza 4 bytes por cada instrucción (las etiquetas y directivas no ocupan espacio), registrando la dirección de la instrucción que sigue a cada etiqueta; ahí mismo se detectan etiquetas duplicadas y directivas inválidas. La segunda pasada (`assemble`) codifica cada instrucción, y para `beq/bne/blt/bge/bltu/bgeu/jal` calcula el offset como `labels[etiqueta] − dirección_de_esta_instrucción`, lo que permite referencias tanto hacia adelante (la etiqueta aún no aparece en el código, pero su dirección ya se conoce gracias a la primera pasada) como hacia atrás. Una etiqueta usada pero nunca definida produce un error explícito en vez de una dirección basura.

3.4 Validación y mensajes de error

Cada error se reporta como un diagnóstico con archivo, línea, columna y el fragmento de código subrayado (estilo compilador), vía la clase `AssemblerError`. El assembler detecta y rechaza, entre otros:

- Mnemónico desconocido.
- Número de operandos incorrecto para el formato de la instrucción.
- Registro fuera de rango para RV32E (x16 en adelante, o nombres ABI como s2/a6 que existen en RV32I pero no en RV32E).
- Inmediato que no cabe en el campo con signo/sin signo correspondiente (por ejemplo, un inmediato de `addi` fuera de $[-2048, 2047]$).
- Offset de salto/branch impar, o fuera del rango alcanzable (±4 KiB para branches, ±1 MiB para `jal`).

- Etiqueta duplicada o indefinida.
- Directiva no soportada (por ejemplo `.data` o `.word`), para no ignorar en silencio algo que cambiaría la imagen de memoria.

Además, usar una instrucción de corrimiento (`sll/srl/sra/slli/srli/srai`) no es un error, porque son válidas en RV32E, pero genera un `AssemblerWarning` recordando que en este núcleo se ejecutan como ADD.

3.5 Ejemplo de traducción: assembly → código máquina

Tomando dos líneas reales de `game.s` (con `s0 = x8` y `a4 = x14`):

Dirección	Assembly	Campos codificados	Palabra (hex)
`0x00000000`	`lui s0, 0x80000`	`imm20=0x80000, rd=8, opcode=0x37`	`80000437`
`0x00000068`	`sw a4, 4(s0)`	`imm=4 → imm[11:5]=0, imm[4:0]=4; rs2=14 (a4), rs1=8 (s0), funct3=2, opcode=0x23`	`00e42223`

Para el segundo caso: `0<<25 | 14<<20 | 8<<15 | 2<<12 | 4<<7 | 0x23 = 0x00E42223`. Ambas palabras aparecen tal cual en `sw/game.hex`, en las líneas 1 y 27 (direcciones 0x00 y 0x68), y la prueba de regresión de §3.6 verifica que todo `game.hex` coincide con lo que produce el assembler a partir de `game.s`.

3.6 Pruebas

Se agregó una batería de pruebas automatizadas en `sw/test_assembler.py`, usando únicamente `unittest` de la librería estándar de Python (sin dependencias externas, sin tocar ningún assembler de RISC-V). Se ejecuta con:

```
$ cd sw && python3 -m unittest test_assembler.py -v
```

Las 23 pruebas cubren cinco frentes, y todas pasan (OK) contra la versión actual de `assembler.py` y `game.s`:

- **Codificación por formato (9 pruebas):** R, I (positivo y negativo/complemento a 2), S, B (con reconstrucción del offset disperso en los bits 31/25-30/8-11/7 para confirmar el round-trip), U y J, comparando la palabra producida contra el valor calculado a mano con la fórmula del formato, además de casos borde como un offset de branch impar.
- **Parseo de operandos (5 pruebas):** registros por nombre ABI y por `xN`, rechazo de registros fuera de RV32E, inmediatos en decimal/hex/binario, y operandos de memoria `offset(registro)`.
- **Validación / manejo de errores (7 pruebas):** instrucción desconocida, inmediato fuera de rango, etiqueta duplicada, etiqueta indefinida, registro fuera de RV32E, directiva no soportada, y que un corrimiento (`slli`) produzca un warning y no un error.
- **Etiquetas y branches (1 prueba):** un programa con una etiqueta hacia adelante y otra hacia atrás, verificando que ambos offsets calculados coincidan con la distancia real en bytes.
- **Regresión de extremo a extremo (1 prueba):** ensambla el `sw/game.s` real y compara, línea por línea, el resultado contra el `sw/game.hex` del repositorio. Es la verificación que confirma que el binario cargado en la FPGA corresponde exactamente al assembly documentado, y se debe repetir cada vez que se modifica `game.s`.

```
Ran 23 tests in 0.003s

OK
```

4. Validación en la FPGA y ajustes finales

Además de las pruebas del assembler, el sistema se validó en tres niveles: (1) simulación RTL del juego completo con `sim/tb_game_smoke.v` (Icarus Verilog con los modelos de celdas iCE40 de Yosys), (2) simulación del netlist ya sintetizado, y (3) pruebas manuales en la Go Board. Las pruebas en la placa revelaron un problema de hardware y dos problemas de usabilidad que no se veían en simulación. También motivaron un cambio en cómo se mide el tiempo. Esta sección documenta cada uno.

4.1 Reset de encendido: la CPU arrancaba antes que la RAM

Síntoma: al programar la placa, los displays mostraban “00” y ningún LED se encendía, aunque la simulación RTL y la del netlist sintetizado mostraban los 4 LEDs encendidos a los pocos ciclos. “00” es justamente el valor de reset del registro de dígitos, así que el síntoma indicaba que la CPU nunca llegaba a escribir en los periféricos.

Causa: el reset de encendido original de `pochoco_soc.v` mantenía la CPU en reset sólo 15 ciclos (0,6 µs). En la iCE40, los bloques de RAM (que aquí contienen tanto el programa como el banco de registros del Espino Core) pueden no entregar lecturas válidas durante un intervalo corto tras la configuración. Si la CPU lee basura en lugar de `lui s0, 0x80000`, `s0` queda en 0 y todas las escrituras “a periféricos” terminan en la RAM, lo que coincide con el síntoma. La simulación no lo muestra porque modela RAM ideal.

Solución: se alargó el reset a 2^{12} ciclos (~164 µs). Una primera versión con un contador de 16 bits y comparación completa no cupo en la FPGA (1284/1280 celdas lógicas), así que se usa un contador de 13 bits cuyo bit más significativo es directamente la señal de fin de reset, sin comparador:

```
reg  [12:0] por_cnt = 13'b0;
wire        rst_ni;
always @(posedge clk) begin
  if (!por_cnt[12]) por_cnt <= por_cnt + 13'd1;
end
assign rst_ni = por_cnt[12];
```

Con este cambio el juego arrancó correctamente en la placa.

4.2 Tiempo de cada ronda visible en pantalla

En la primera versión, tras una ronda correcta `show_decimal` escribía el tiempo en los displays y el programa saltaba de inmediato a `round_start`, cuya primera instrucción (`sw x0, 0(s0)`) limpia el display. El tiempo de cada ronda quedaba visible sólo unos ciclos, y en la placa únicamente se alcanzaba a ver el promedio final. Se agregó una espera de 2 s (`hold_result`, 50 000 000 ciclos medidos con el contador) después de mostrar el resultado y antes de pasar a la siguiente ronda o al promedio. Así, el tiempo de la décima ronda también se alcanza a leer.

4.3 Reinicio de la partida con un botón

La Go Board no tiene botón de reset conectado al diseño, y originalmente el programa terminaba en un bucle infinito (`halt: jal x0, halt`), por lo que la única forma de jugar otra vez era desconectar la placa. Ahora, al terminar las 10 rondas, el promedio queda en pantalla y el programa espera primero a

que se suelte el botón de la última ronda (para no reiniciar sin querer) y luego a que se presione cualquier botón, momento en que salta a `_start` y comienza una partida nueva con contadores en cero y una semilla nueva:

```
halt:
    lw   t0, 8(s0)
    bne  t0, x0, halt          # espera a que se suelte el boton
wait_restart:
    lw   t0, 8(s0)
    beq  t0, x0, wait_restart  # espera una pulsacion
    jal  x0, _start            # partida nueva
```

4.4 Cronómetro iniciado al apagarse los 4 LEDs

La primera versión leía `cyc_start` al comienzo de la ronda, antes de encender los 4 LEDs, siguiendo una lectura literal del enunciado (“la medición de tiempo arranca en el mismo instante lógico en que comienza la ronda”). En consecuencia, todos los tiempos mostrados incluían los 3 s fijos de espera: un jugador que reaccionaba en 0,4 s veía “34”, y ningún resultado podía ser menor que 30. Al probarlo en la placa, esto resultó poco informativo, porque el número mostrado era casi todo espera fija y no reflejaba el reflejo del jugador.

En la versión final, el cronómetro arranca en el instante en que se apagan los 4 LEDs y queda encendido sólo el LED objetivo: se agregó una lectura del contador a `a2` justo después de `sw a4, 4(s0)` (§2.2). La lectura al comienzo de la ronda se mantiene, pero sólo para controlar la espera de 3 s y alimentar el generador pseudoaleatorio. Ahora los valores mostrados son el tiempo de reacción real: “04” significa 0,4 s. Como la conversión trunca a la décima inferior, una reacción muy rápida puede mostrarse como “00” o “01”. Si se quisiera volver al comportamiento original, basta con eliminar esa segunda lectura de a2.

4.5 Uso de recursos y temporización

Recurso	Uso	Disponible	%
Celdas lógicas (ICESTORM_LC)	1259	1280	98 %
Bloques de RAM (ICESTORM_RAM)	12	16	75 %
Pines de E/S (SB_IO)	27	72	37 %

La frecuencia máxima reportada por `nextpnr-ice40` es ~44,7 MHz, holgada frente a los 25 MHz del reloj de la placa. El diseño ocupa el 98 % de la lógica de la HX1K, por lo que cualquier funcionalidad nueva (como las de §4.2 a §4.4) se implementó en software, que sólo consume palabras de RAM (98 de 512) y no celdas lógicas.

4.6 Pruebas en la placa

Lista de verificación usada al probar el juego en la Go Board:

Prueba	Resultado esperado
Encendido	Los 4 LEDs se encienden ~3 s y luego queda uno solo.
Botón correcto	Se muestra el tiempo de reacción en décimas durante 2 s y comienza la ronda siguiente.
Botón incorrecto	“EE” ~0,5 s, LEDs apagados, y se repite la misma ronda sin perder el progreso.
Botón mantenido desde la ronda anterior	No cuenta como pulsación hasta soltarlo.
Selección del LED objetivo	Varía entre rondas sin un patrón evidente.
10 rondas correctas	Se muestra el promedio en décimas y queda fijo en pantalla.
Reinicio	Con el promedio en pantalla, cualquier botón inicia una partida nueva.

Para reproducir: `make clean && make all` en la raíz del repositorio sintetiza y programa la placa (en WSL, la placa debe conectarse con `usbipd attach --wsl` y `iceprog` puede requerir `sudo` o una regla udev para el dispositivo FTDI 0403:6010).

5. Conclusiones

Se implementó el juego de reflejos completo en assembly RV32E sobre el Pochoco SoC, ensamblado con un assembler propio y ejecutado en la Go Board. Todo el comportamiento del juego vive en software: la temporización se basa en un único contador de ciclos mapeado en memoria, y las restricciones de la ALU (sin corrimientos, sin multiplicación ni división) se resolvieron con sumas y restas repetidas, incluida la generación pseudoaleatoria y el despliegue decimal en los 7 segmentos.

La validación en hardware fue indispensable: el problema del reset de encendido (§4.1) no aparecía en ninguna simulación, porque éstas modelan RAM ideal, y sólo se detectó al programar la placa. Del mismo modo, los ajustes de usabilidad de §4.2–§4.4 surgieron de jugar en la placa y no de las especificaciones. Finalmente, trabajar con el 98 % de la lógica de la HX1K ocupada obligó a preferir soluciones en software antes que en hardware, y a escribir el hardware necesario (como el reset) de la forma más económica posible.